

CS & IT ENGINEERING

DATA STRUCTURE

Tree (Part-III)



JOIN TELEGRAM TO ENROLL

SHAIENDRA SIR

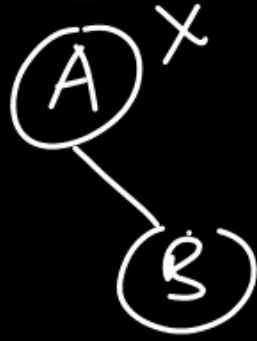
Binary Tree

- Complete Binary Tree
- Perfect Binary Tree
- Binary Tree Traversal
- Binary Tree Construction

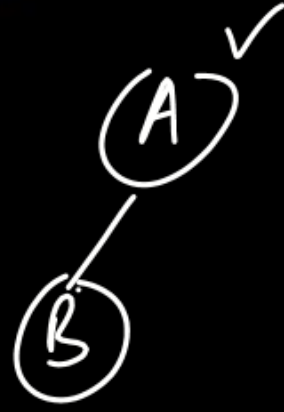
Complete Binary Tree: (CBT):



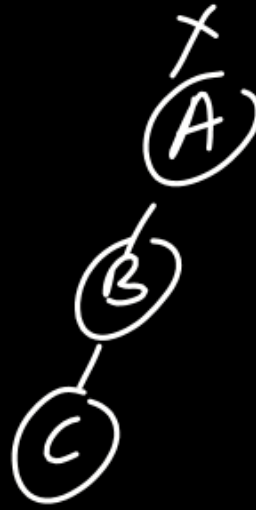
CBT ✓
SBT ✓



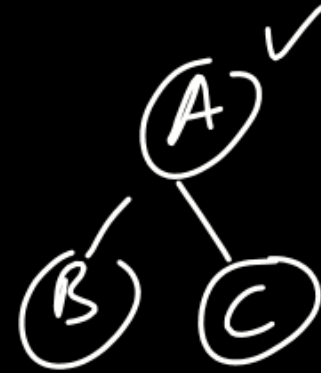
CBT ✗
SBT ✗



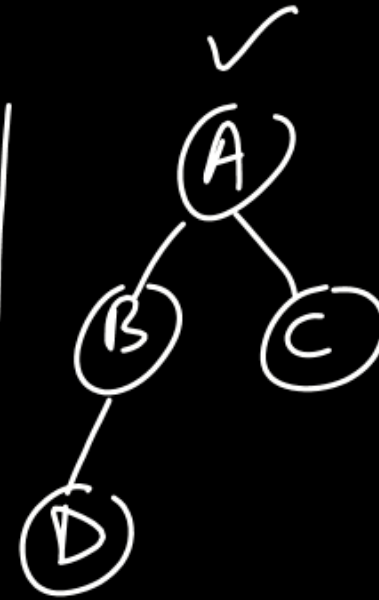
CBT ✓
SBT ✗



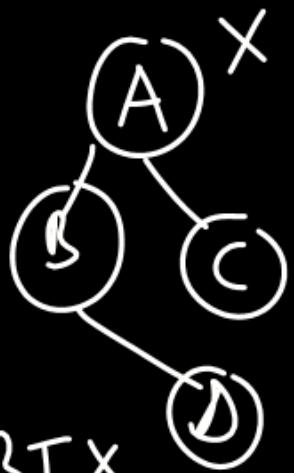
CBT ✗
SBT ✗



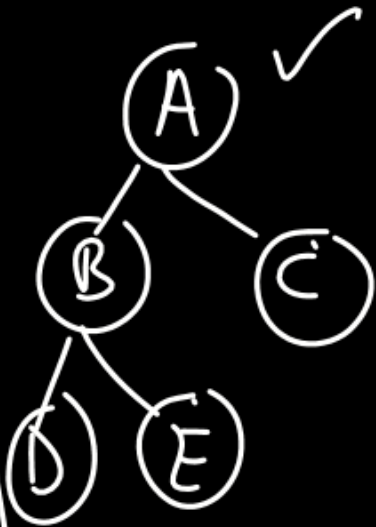
CBT ✓
SBT ✓



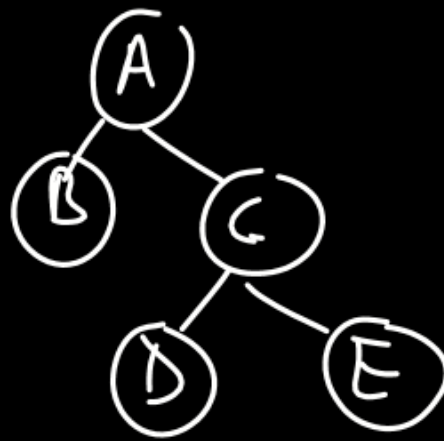
CBT ✓
SBT ✗



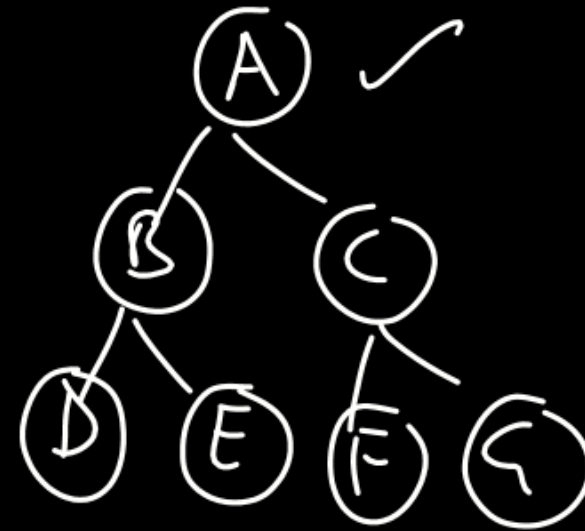
CBT ✗
SBT ✗



CBT ✓
SBT ✓



CBT ✗
SBT ✓

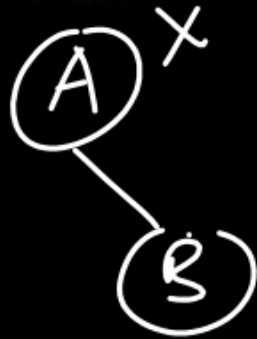


CBT ✓
SBT ✓

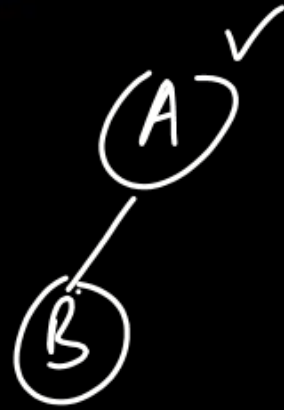
Complete Binary Tree: (CBT):



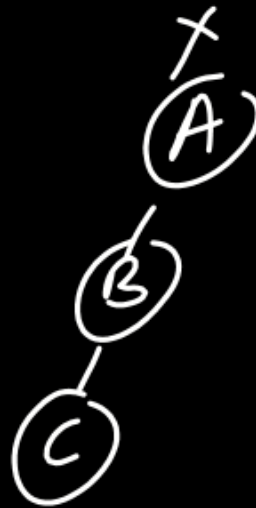
CBT ✓
SBT ✓



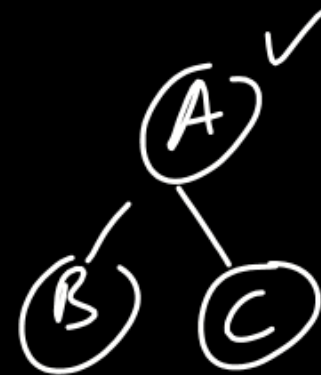
CBT x
SBT x



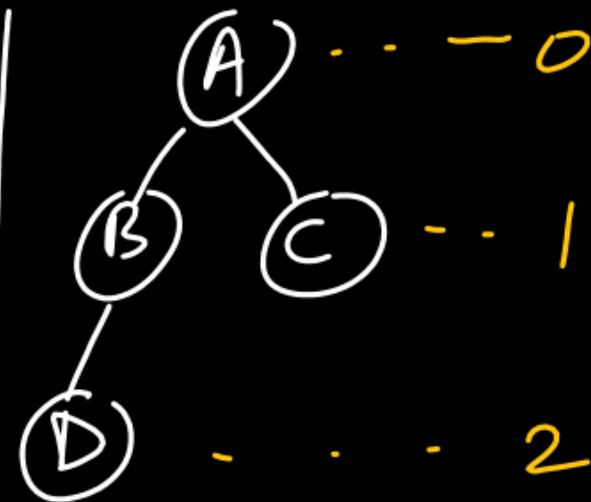
CBT ✓
SBT x



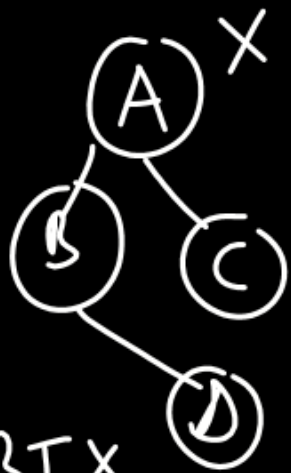
CBT x
SBT x



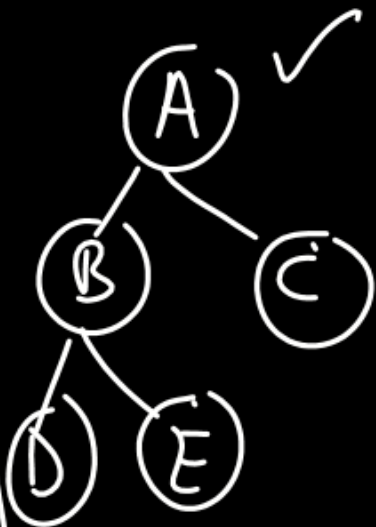
CBT ✓
SBT ✓



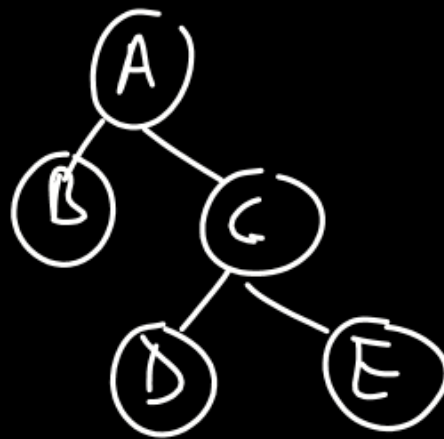
CBT ✓
SBT x



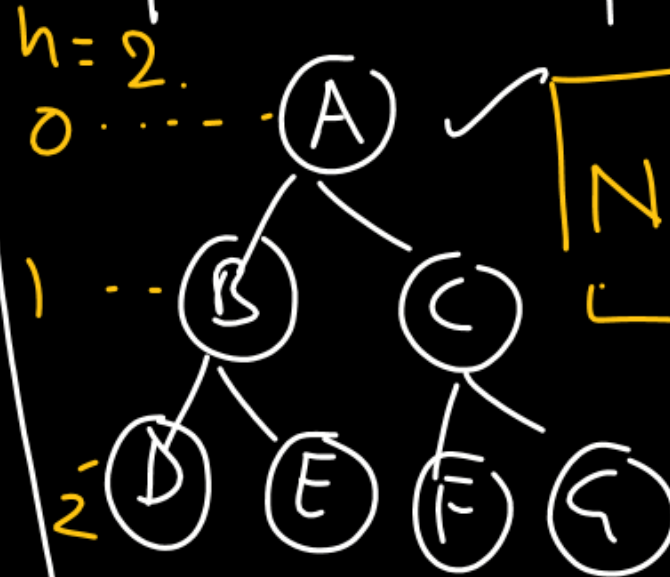
CBT x
SBT x



CBT ✓
SBT ✓



CBT x
SBT ✓



$$N = 2^{h+1} - 1$$

CBT ✓
SBT ✓

$$2^h \leq N \leq 2^{h+1} - 1$$

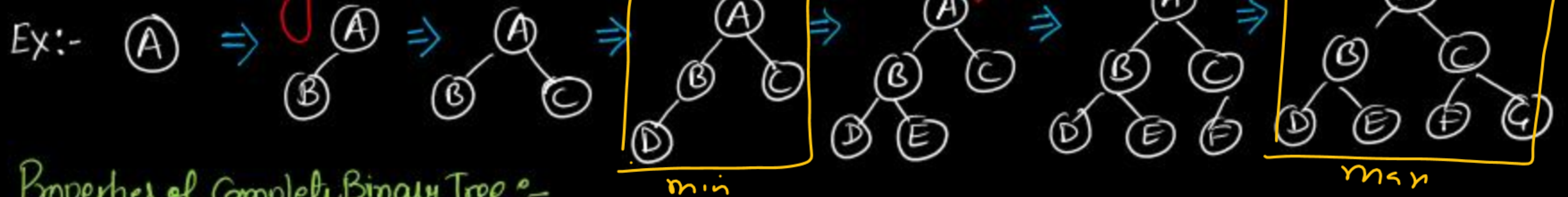
$$\log_2(N+1) - 1 \leq h \leq \log_2 N$$

Complete Binary Tree

All levels are completely filled except possibly the last level, which is filled from left to right.

Used in Heap Data Structure.

- In Complete Binary Tree, the node should be added from Left



Properties of Complete Binary Tree :-

- The max. no. of nodes in Complete Binary tree = $2^{h+1} - 1$
- The mini no. of nodes in Complete Binary tree = 2^h
- The mini height of a Complete Binary tree = $\log_2(n+1) - 1$
- The max height of a Complete Binary tree = $\log_2 n$

Representation of CBT in memory:-

Index
Start from 1

Algo: Parent of (i)

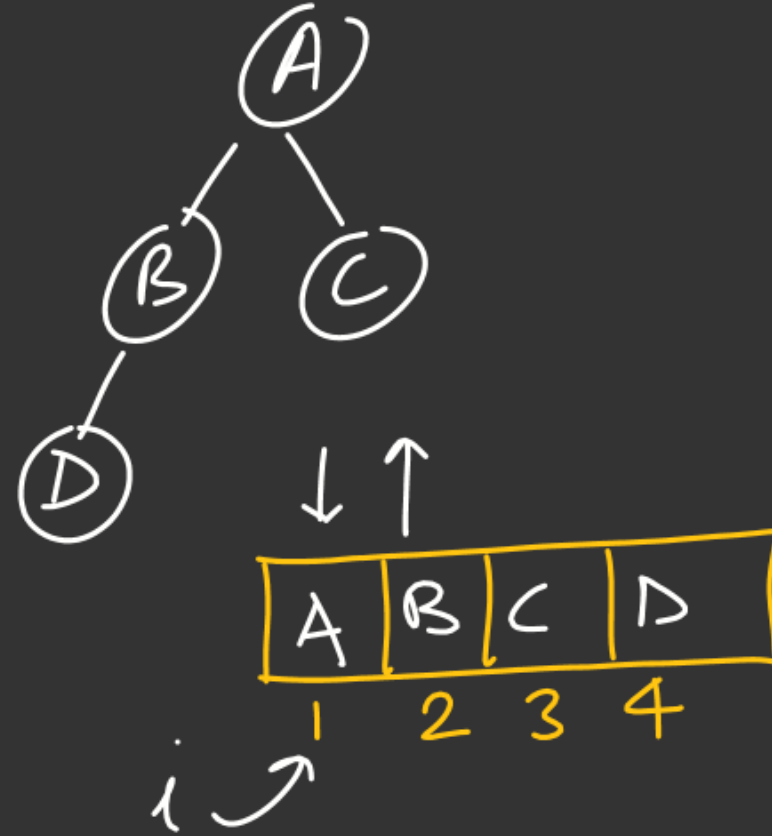
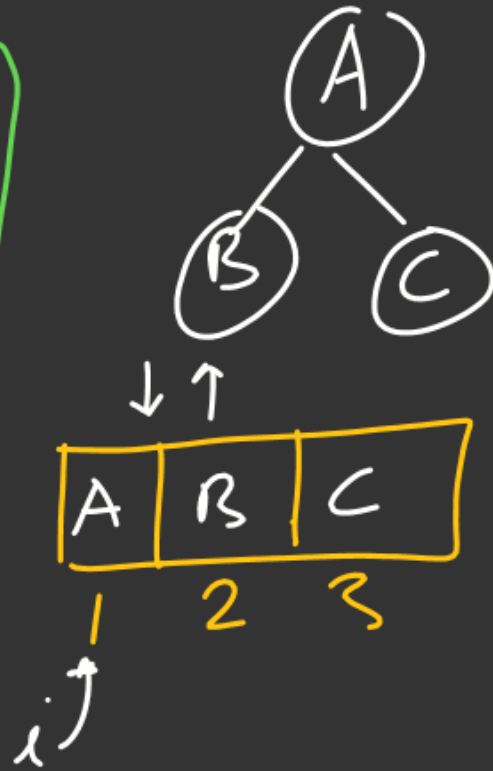
- return $\lfloor i/2 \rfloor$

left (i)

- return $2i$

Right (i)

- return $2i+1$



| Array | $\left\{ \begin{array}{l} \text{max size of Array} = n \\ \text{min size of Array} = n \end{array} \right\}$ } Array Representation
Good for CBT

An array X of n distinct integers is interpreted as a complete binary tree. The index of the first element of the array is 0. The index of the parent of element $X[i]$, $i \neq 0$, is?

A. $\left\lfloor \frac{i}{2} \right\rfloor = 3 \times$

B. $\left\lfloor \frac{i-1}{2} \right\rfloor = 3 \times$

C. $\left\lfloor \frac{i}{2} \right\rfloor = 3 \times$

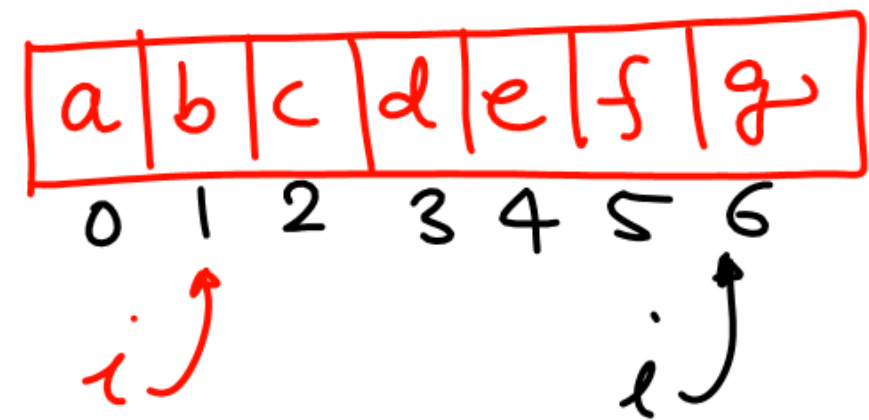
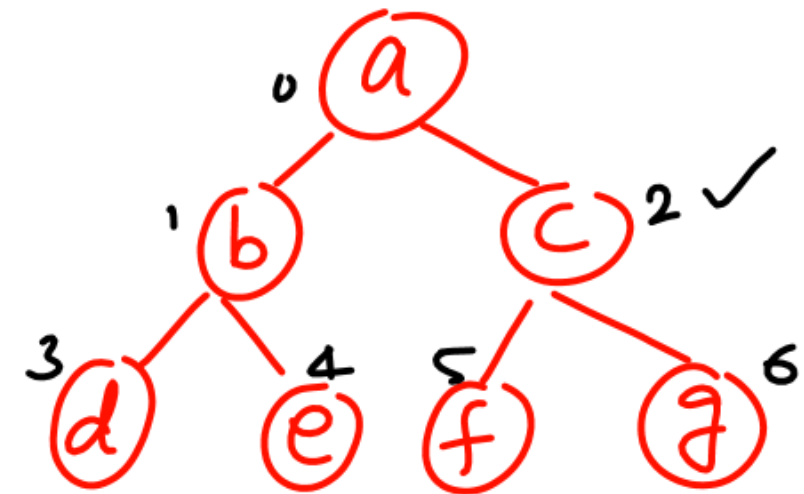
✓ D. $\left\lfloor \frac{i}{2} \right\rfloor - 1 = 2 \checkmark$

Parent(6): $\Rightarrow (2)$

Parent(i)
- return $\left\lfloor \frac{i}{2} \right\rfloor - 1$

left(i)
- return ?

Right(i)
- return ?



Consider a complete binary tree represented using an array of n elements stored from index 0 to $n-1$. For any valid index i such that $0 \leq i < n$, let:

$P(i)$ denote the index of the parent of the node at index i

$L(i)$ denote the index of the left child

$R(i)$ denote the index of the right child

Which one of the following correctly defines $P(i)$, $L(i)$, and $R(i)$ for such an array-based binary heap?

A)

- $P(i) = \lfloor \frac{i}{2} \rfloor$
- $L(i) = 2i$
- $R(i) = 2i + 1$

B)

- $P(i) = \lfloor \frac{i-1}{2} \rfloor$
- $L(i) = 2i + 1$
- $R(i) = 2i + 2$

C)

- $P(i) = \lfloor \frac{i+1}{2} \rfloor$
- $L(i) = 2i + 1$
- $R(i) = 2i + 2$

D)

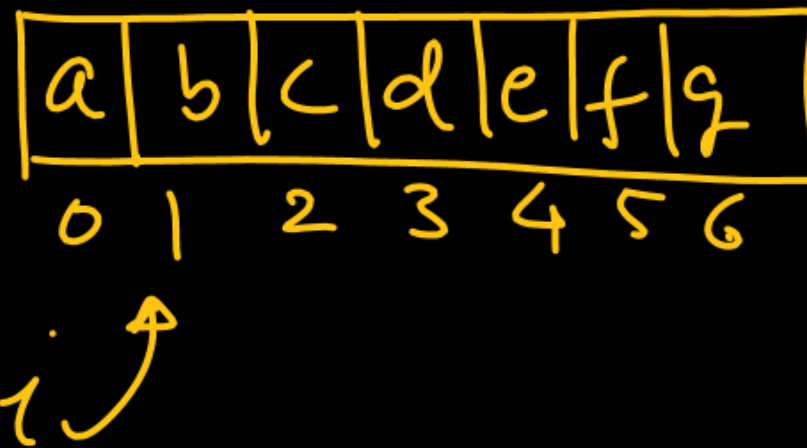
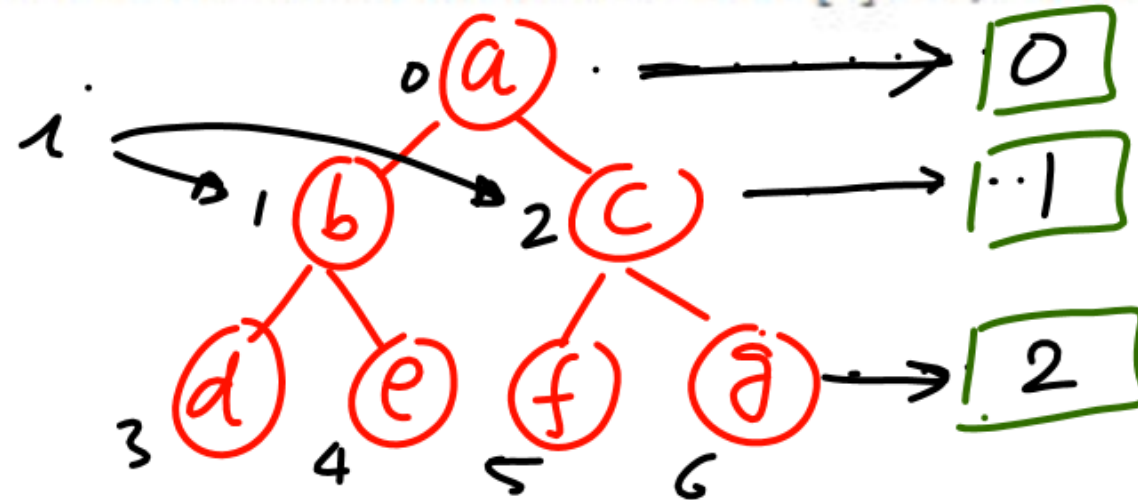
- $P(i) = \lfloor \frac{i}{2} \rfloor - 1$
- $L(i) = 2i - 1$
- $R(i) = 2i$

Question:

An array X of n distinct integers is interpreted as a complete binary tree. The index of the first element of the array is 0. If the root node is at level 0, the level of element $X[i]$, $i \neq 0$, is:

Options:

- ☒ A. $\lfloor \log_2 i \rfloor$ ✗
- ☒ B. $\lceil \log_2(i + 1) \rceil = 1$
- ☒ C. $\lceil \log_2(i + 1) \rceil = 2$
- ☒ D. $\lceil \log_2 i \rceil$ ✗

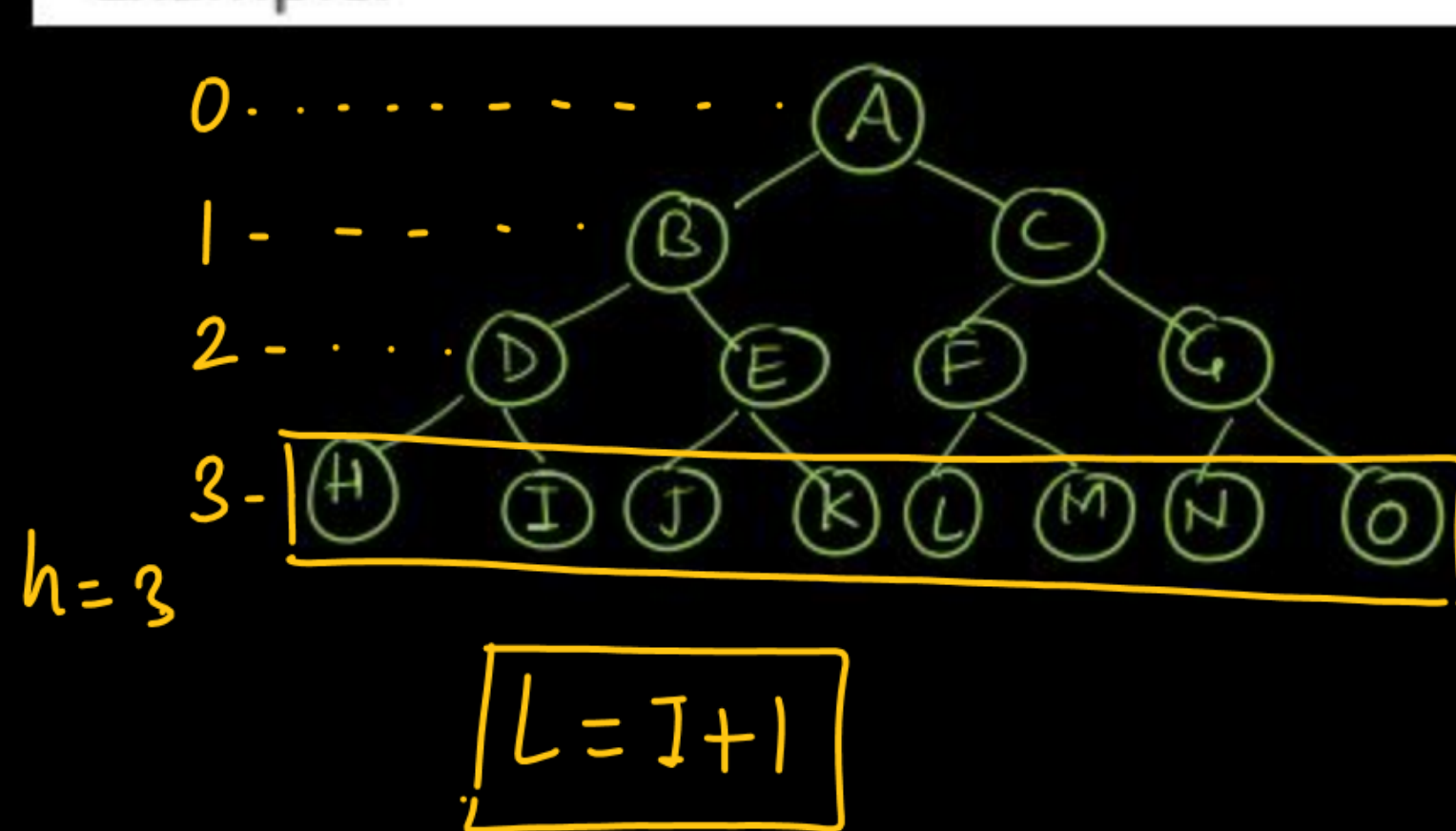


Perfect Binary Tree

All internal nodes have two children, and all leaf nodes are at the same level.

Formula: A perfect binary tree of height h has $2^{h+1} - 1$ nodes.

Example:



$$N \begin{cases} \text{max} = 2^{h+1} - 1 \\ \text{min} = 2^{h+1} - 1 \end{cases}$$

$$I \begin{cases} \text{max} = 2^h - 1 \\ \text{min} = 2^h - 1 \end{cases}$$

$$L \begin{cases} \text{max} = 2^h \\ \text{min} = 2^h \end{cases}$$

(All levels are completely filled, and all leaf nodes are at the same depth.)

Perfect Binary Tree:-

$$N = 2^{h+1} - 1 \Rightarrow h = \log_2(N+1) - 1$$

$$I = 2^h - 1 \Rightarrow h = \log_2(I+1)$$

$$L = 2^h = I+1 \Rightarrow h = \log_2 L$$

Binary Tree Traversal

Traversal refers to visiting each node of a binary tree in a specific order.

1. Preorder Traversal (Root → Left → Right)

2. Inorder Traversal (Left → Root → Right)

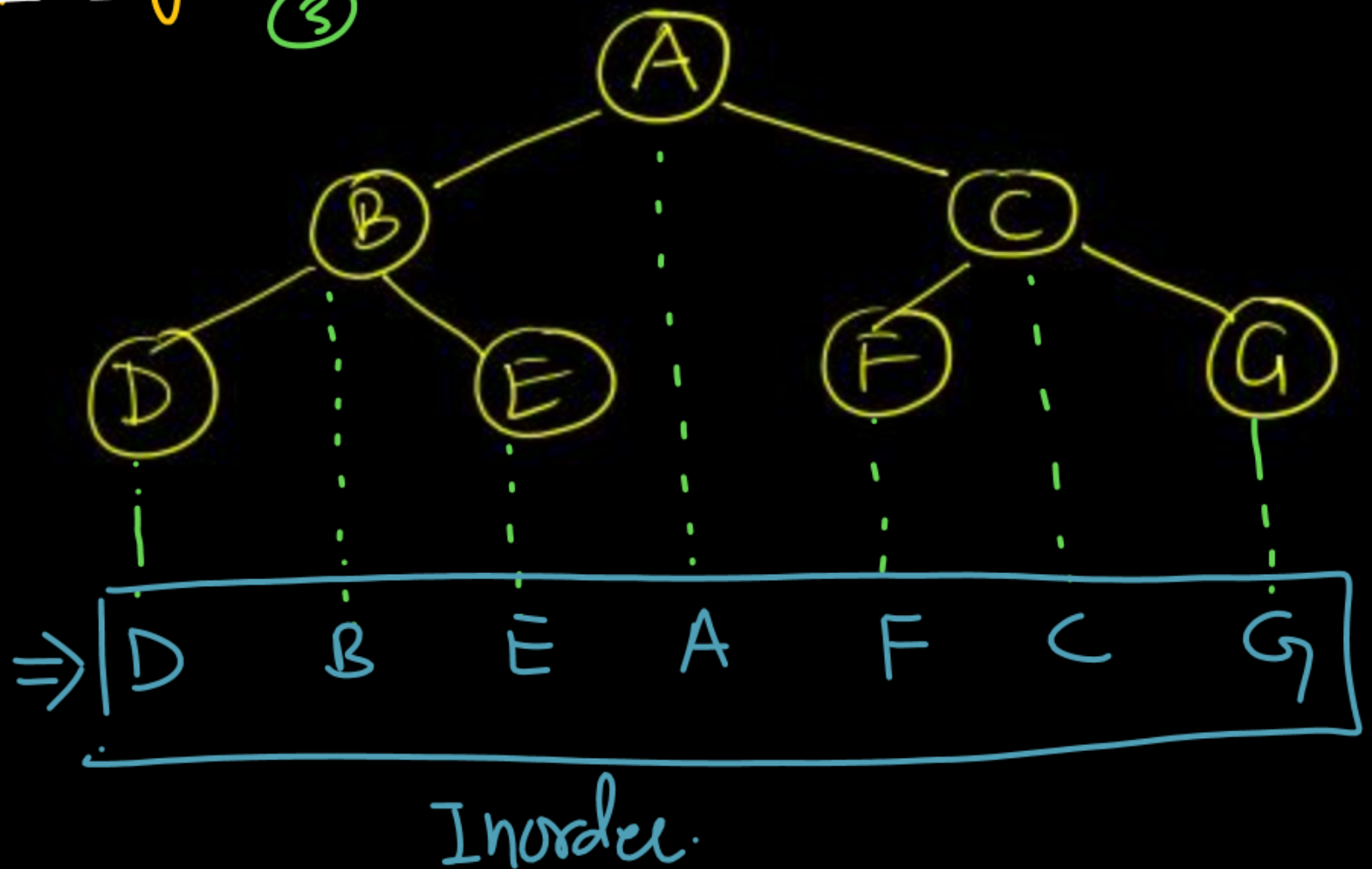
3. Postorder Traversal (Left → Right → Root)

① Root Left Right
② Left Root Right
③ Left Right Root

Preorder: A B D E C F G
 ↑ └─┬─┘ └─┬─┘
 Root left Right

Postorder: D E B F G C A
 └─┬─┘ └─┬─┘ ↑
 left Right Root

Inorder: D B E A F C G
 └─┬─┘ ↑ └─┬─┘
 left Root Right

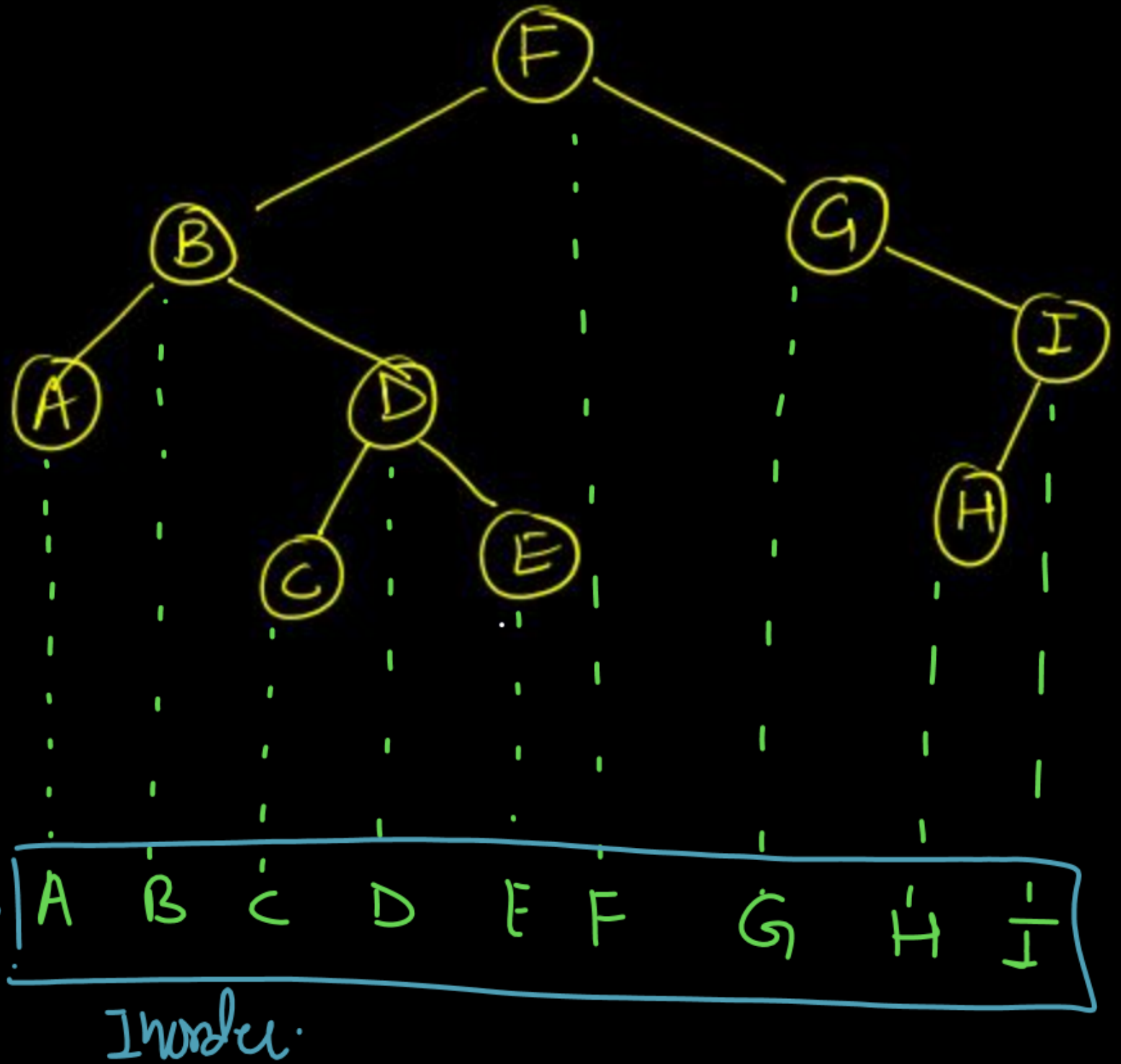


Question:-

Preorder: F B A D C E G I H
 ↑ ↑ ↑ ↑ ↑ ↑
 Root left Right

Postorder: A C E D B H I G F
 ↑ ↑ ↑ ↑ ↑ ↑
 left Right Root

Inorder: A B C D E F G H I
 ↑ ↑ ↑ ↑ ↑ ↑
 left Root Right

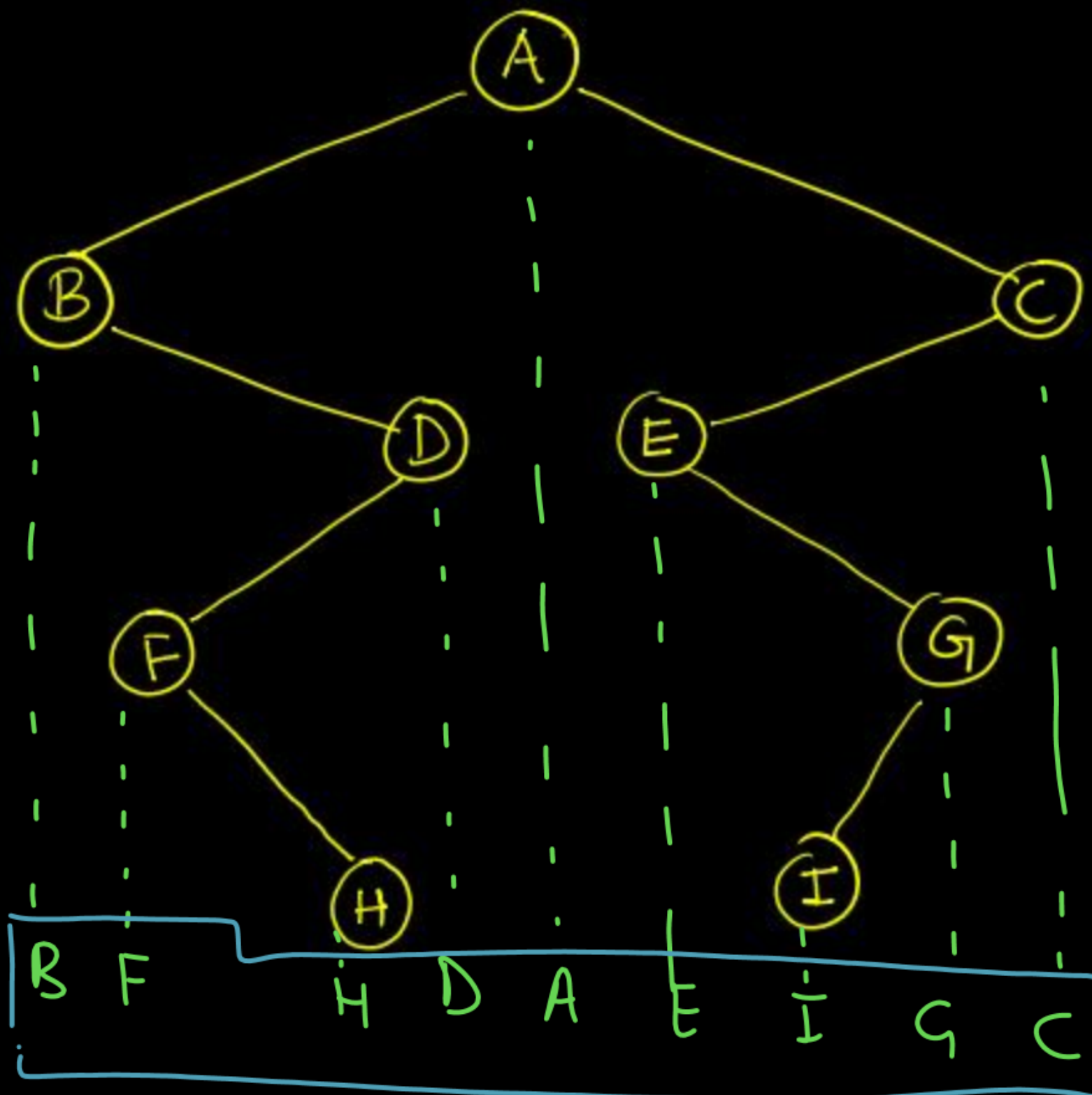


Question:-

Preorder:- A B D F H C E G I

Postorder:- H F D B I G E C A

Inorder:- B F H D A E I G C



Traversal Trick:-

Visit
1st Time.

→ Preorder: A B E F C D

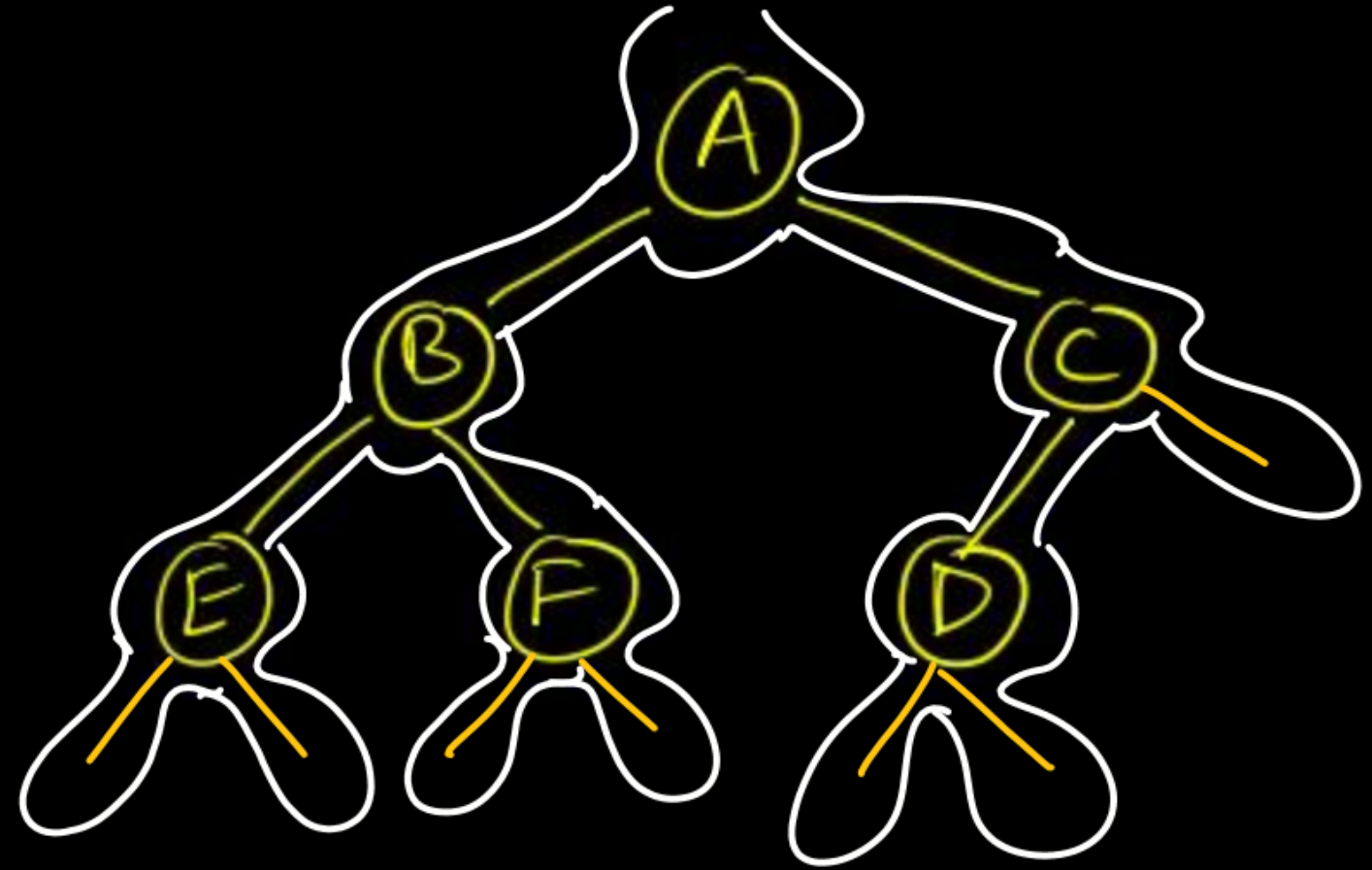
Visit
2nd Time

→ Inorder: E B F A D C

Visit
3rd Time

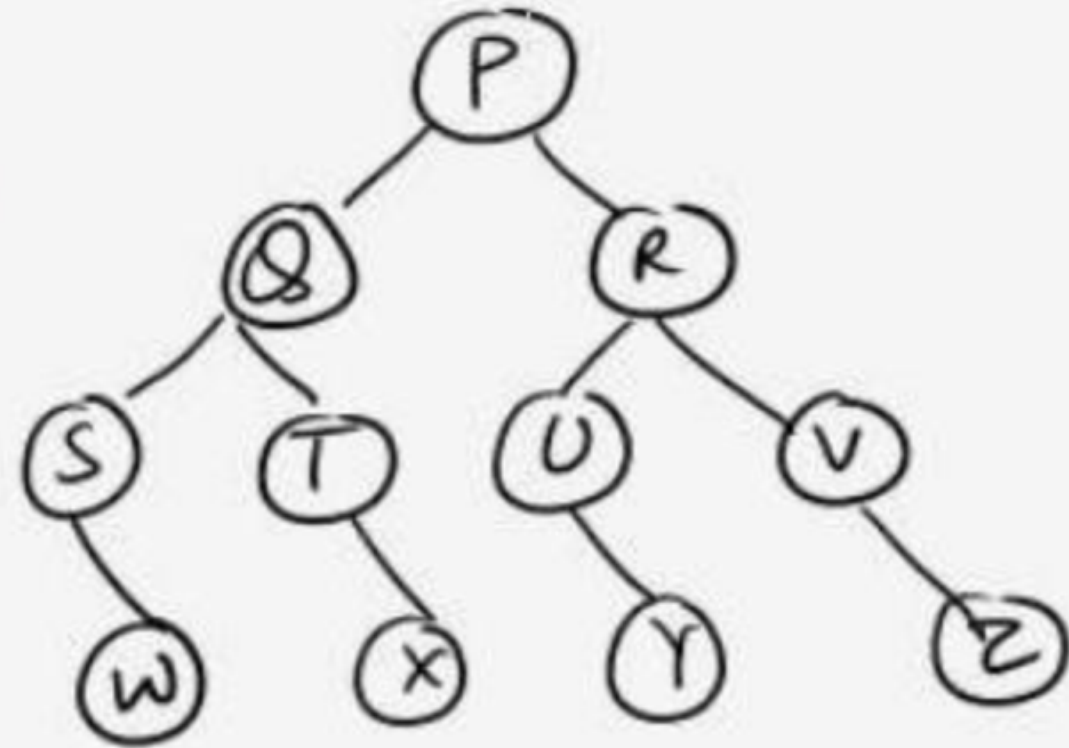
or
Last Time.

→ Postorder: E F B D C A



Question:- Consider the following tree
if the Postorder traversal of the above
tree is $3 - 9 + * 14 + 17 - + /$

Then $SWQXPYVZ = ?$
 $- 3 * 9 / 14 - 17$



$$- 27 / 14 - 17$$

$$- 1 - 17$$

$$\underline{\underline{-18}} \text{ Ans}$$

Postorder: $W \ S \ X \ T \ Q \ Y \ U \ Z \ V \ R \ P$
Given: $3 \ - \ 9 \ + \ * \ 14 \ + \ 17 \ - \ + \ /$

Recursive Code for Traversal:-

$T(n)$ · void Inorder(node *t)



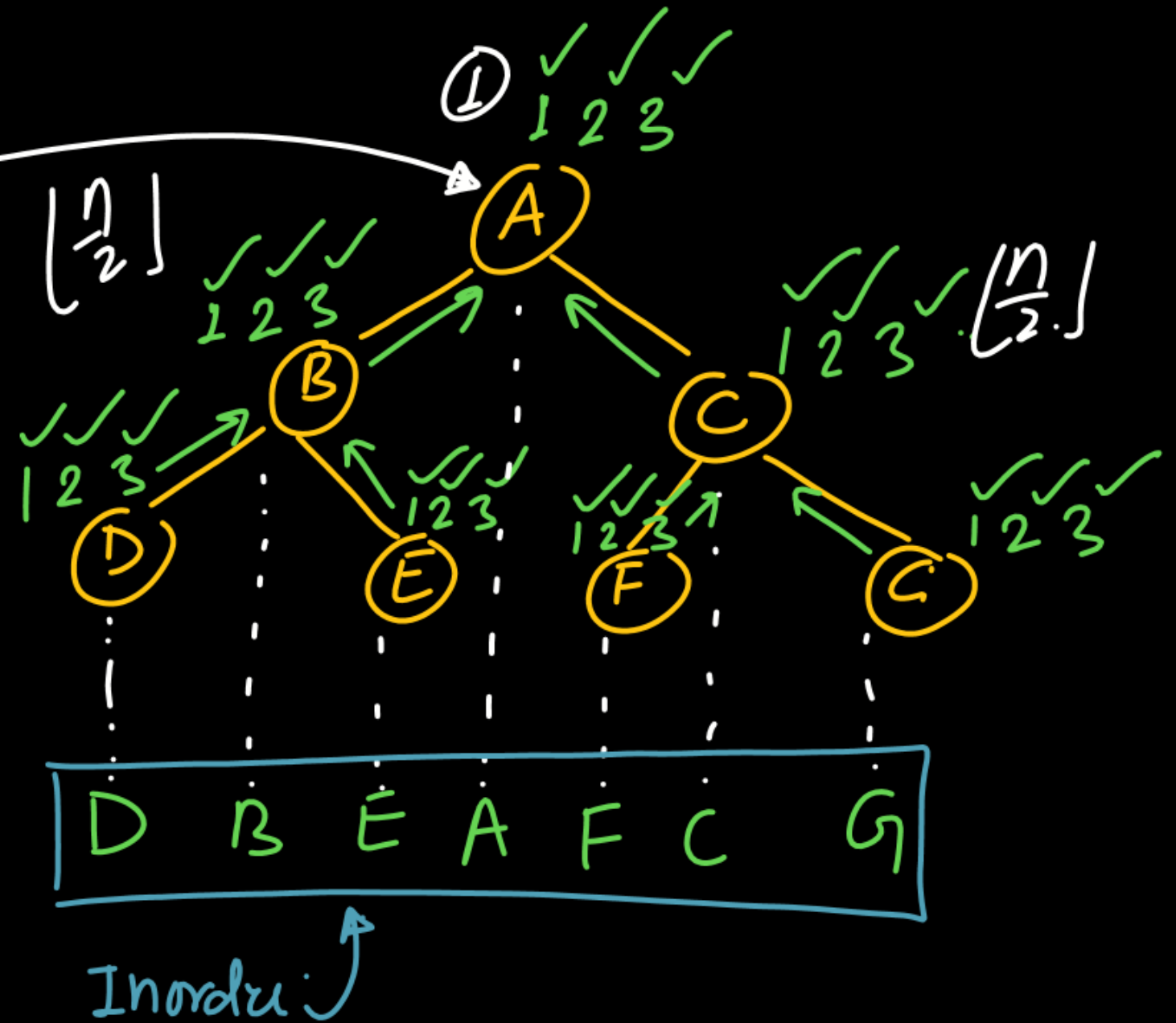
```

{
  if (t)
  {
    ① Inorder(t->left);
    ② printf("%c", t->data);
    ③ Inorder(t->right);
  }
}

```

Recurrence Relation:

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2T(n/2) + 1, & \text{if } n > 1 \end{cases}$$

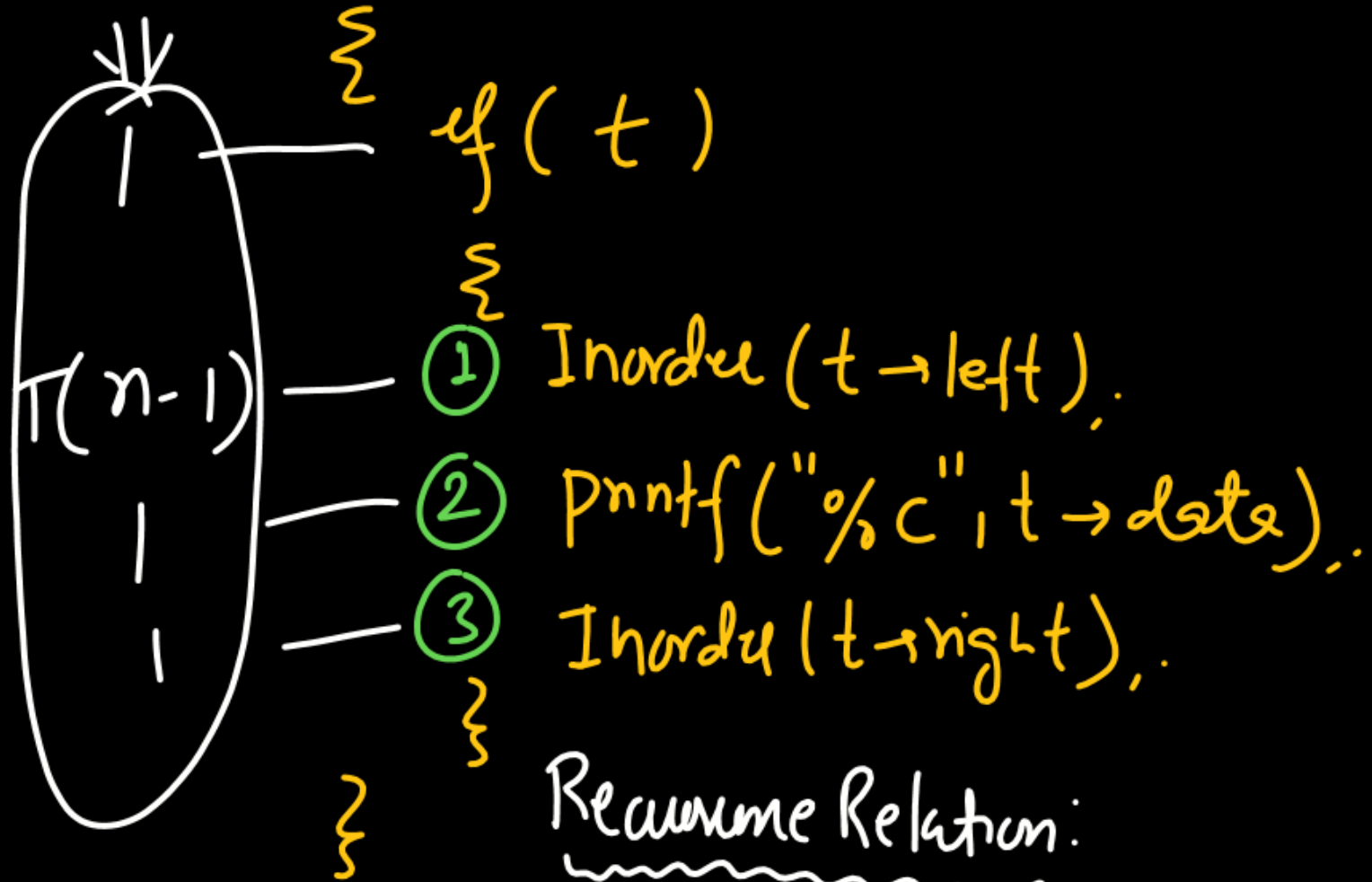


$$T(n) = 2T(n/2) + 1 \Rightarrow \underline{\underline{\theta(n)}}$$

$\swarrow$ BC
 $\swarrow$ WC
 $\swarrow$ AC

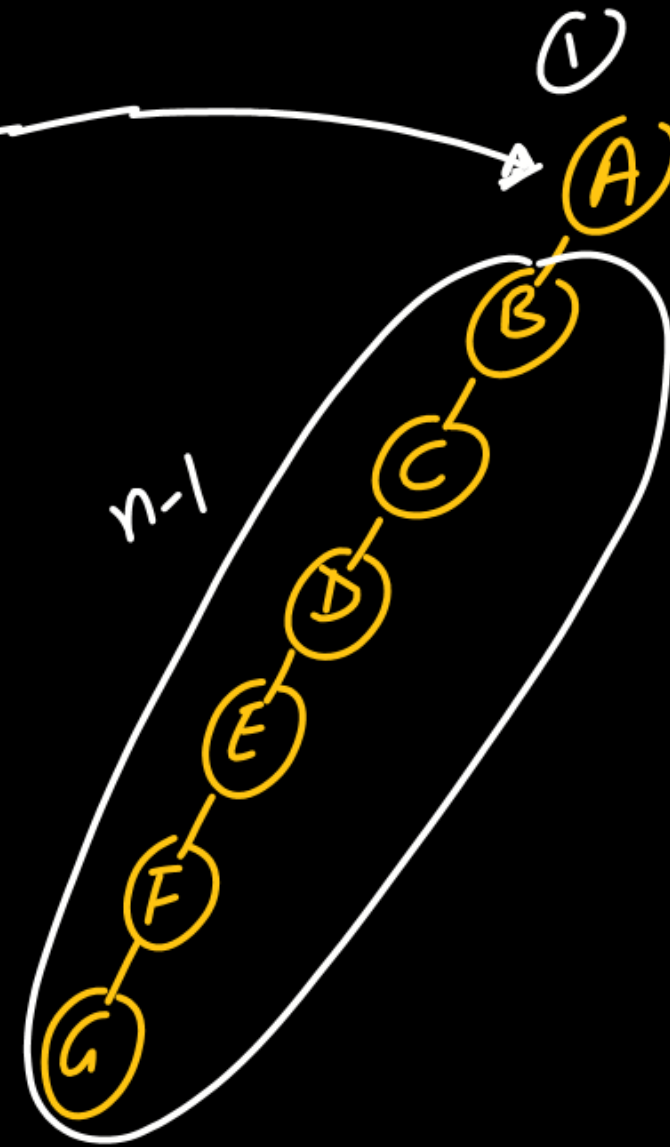
Recursive Code for Traversal:-

$T(n)$. `void Inorder(node *t)`



Recurrence Relation:

$$T(n) = \begin{cases} 1, & \text{if } n=1 \\ T(n-1) + 1, & \text{if } n > 1 \end{cases}$$

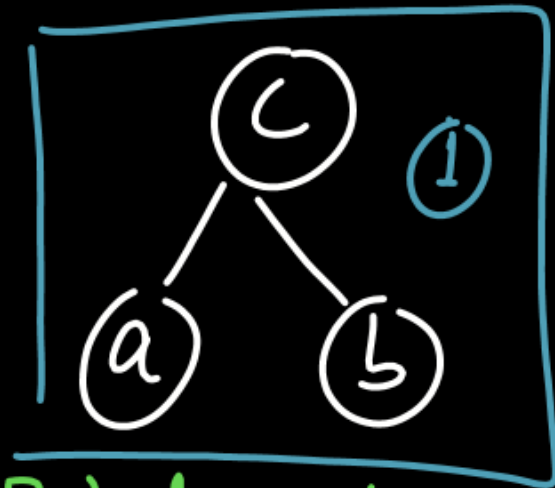


$$T(n) = T(n-1) + 1 \Rightarrow \underline{\underline{\Theta(n)}} \begin{cases} \text{BC} \\ \text{WC} \\ \text{AC} \end{cases}$$

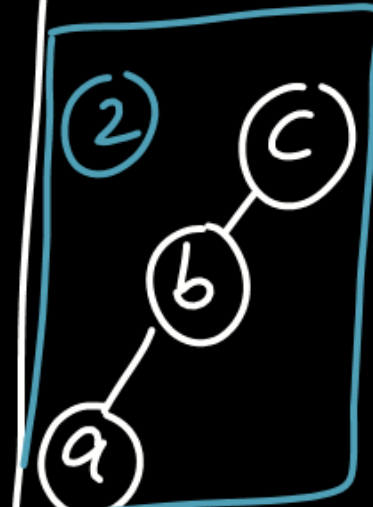
Binary Tree Construction:-

Cases: using Postorder abc only

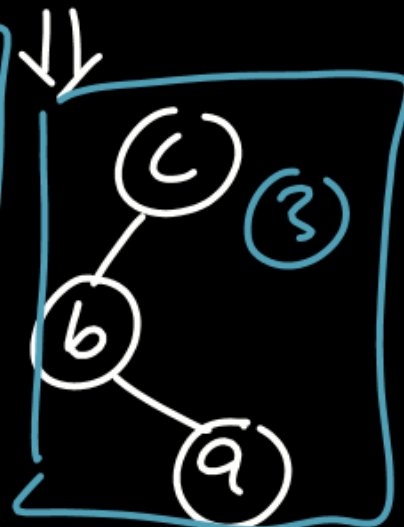
Postorder: Left Right Root
 $\downarrow \quad \downarrow \quad \downarrow$
 $a \quad b \quad c$



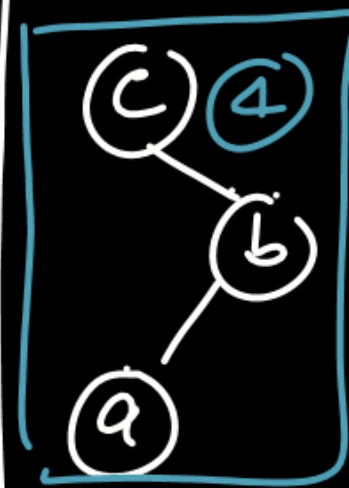
Postorder: abc



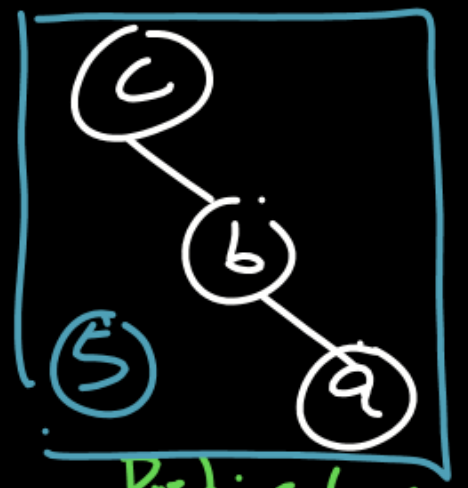
Post: abc



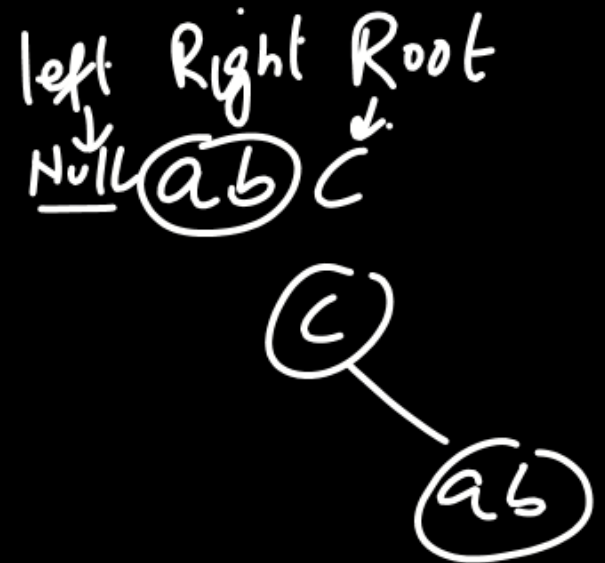
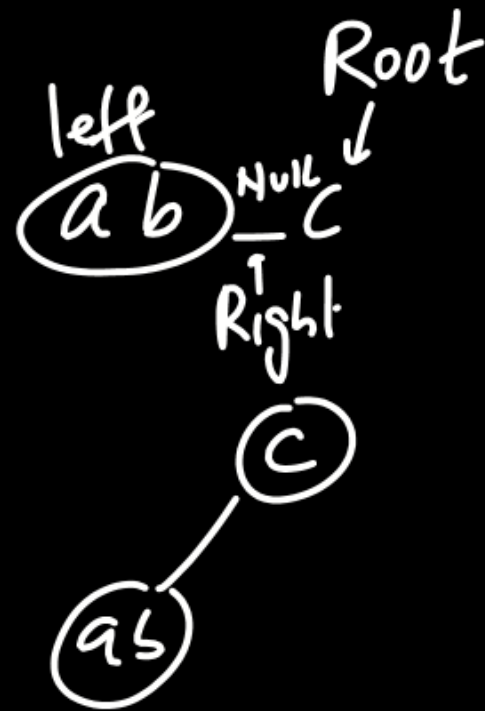
Post: abc



Post: abc



Post: abc

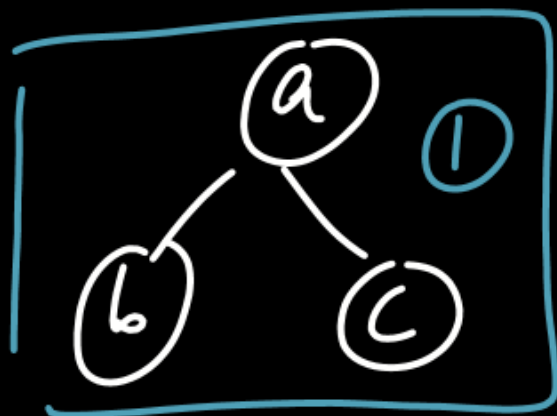


Note:- No. of Binary Tree if only Postorder given = Catalan No =

$$\frac{1}{n+1} \times \frac{2n}{n} C_n$$

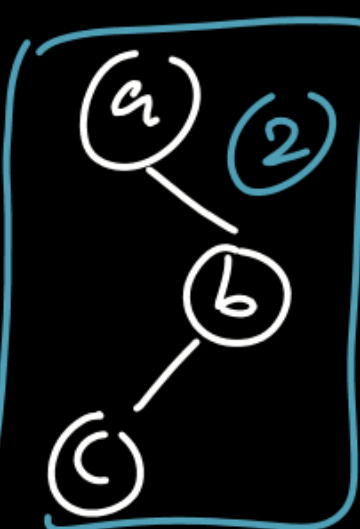
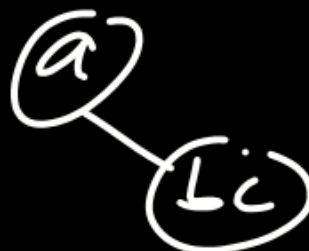
Case 2: using Preorder **abc** only

Preorder: Root left Right
 $\downarrow \quad \downarrow \quad \downarrow$
 a b c

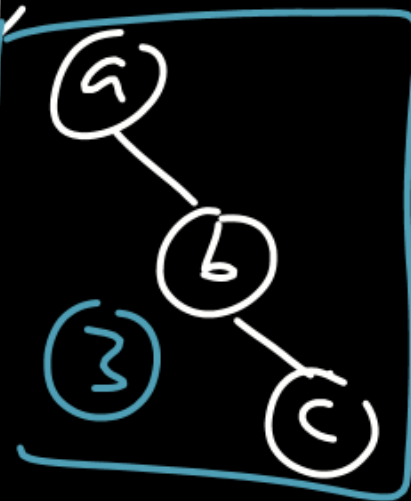


Pre: abc

Root left Right
 a null (bc)

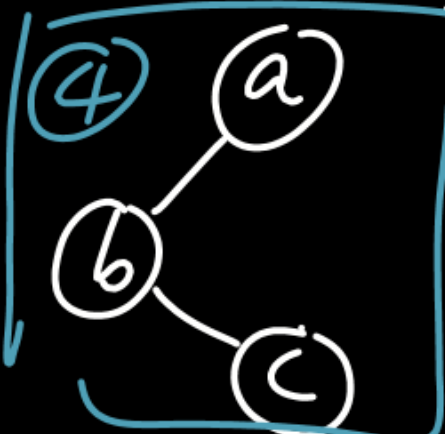
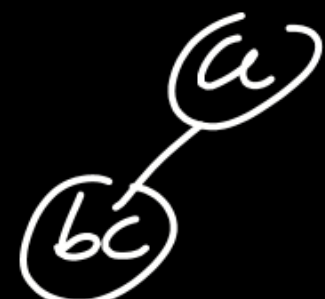


Pre: abc

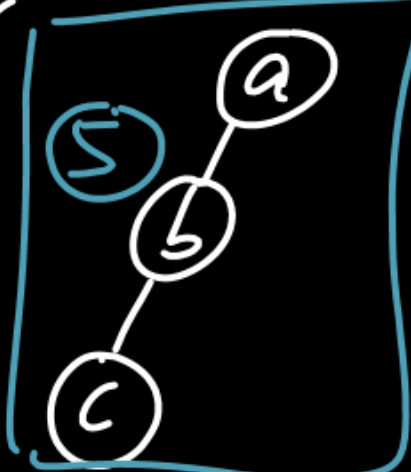


Pre: abc

Root left Right
 a (bc) null



Pre: abc

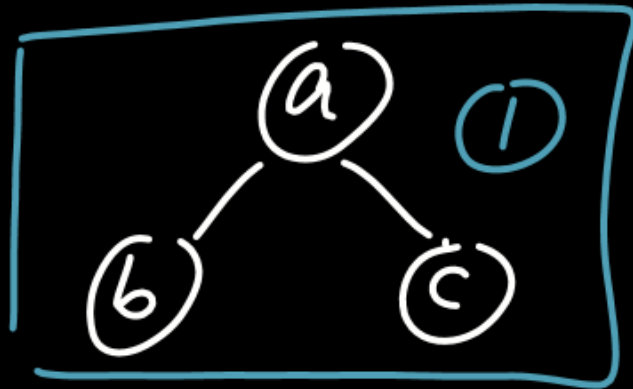


Pre: abc

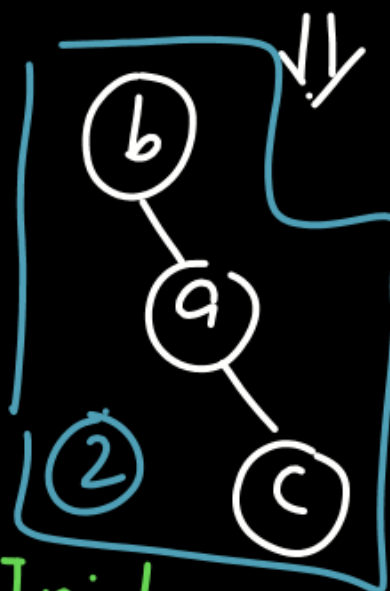
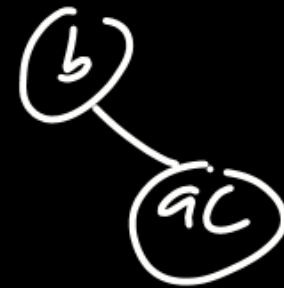
NOTE:- if only Preorder traversal given then No. of Binary Tree = Catalan No. = $\frac{1}{n+1} \times \frac{2n}{C_n}$

Case 3: using inorder bac only

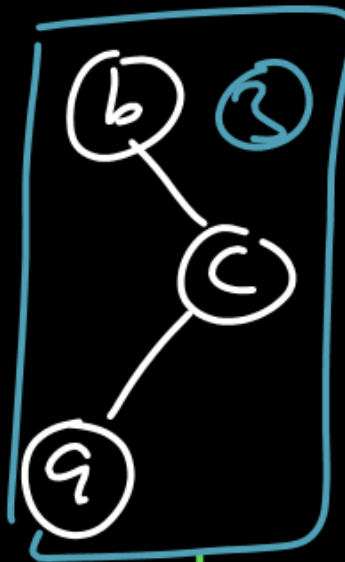
Inorder: Left Root Right



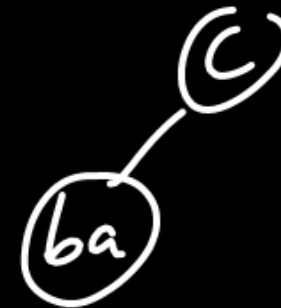
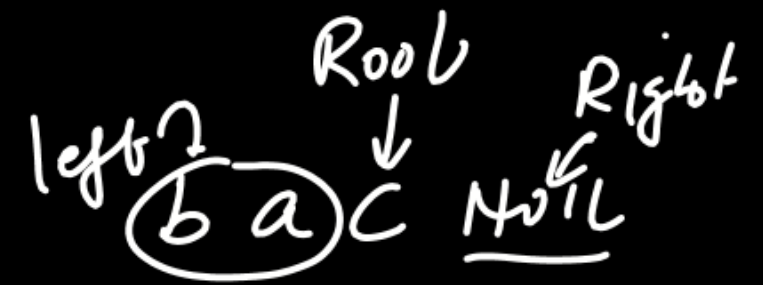
In: bac



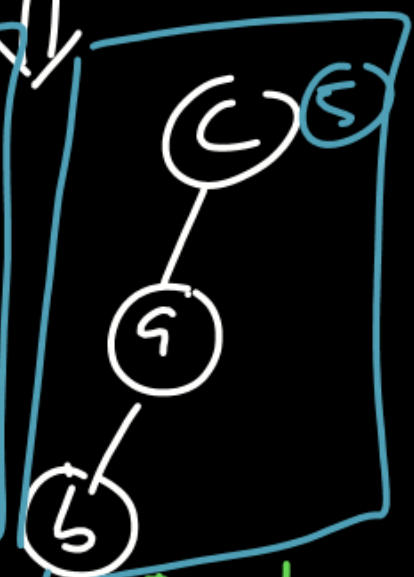
In: bac



In: bac



In: bac



In: bac

NOTE:- if only inorder given Then No. of Binary Tree = Catalan No. = $\frac{1}{n+1} \times \frac{2n}{C_n}$



Shailendra Singh GATE CSE/DA

THANKYOU



GeeksforGeeks GATE